

MISRA C:2012

Regeln & Projektidee

Richtlinien für C-Code in sicherheitskritischen Systemen

Gruppe 3

Andreas · Jonas · Loic · David

5.1

5.2

5.3

5.8

7.2

7.3

8.1

9.1

Übersicht

5.1

Bezeichner

**Externe Namen
unterscheidbar**

Required
entscheidbar · System

5.2

Bezeichner

**Namen im gleichen
Scope unterscheidbar**

Required
entscheidbar · Single TU

5.3

Bezeichner

**Kein Verdecken
(Shadowing)**

Required
entscheidbar · Single TU

5.8

Bezeichner

**Externe Namen
projektweit eindeutig**

Required
entscheidbar · System

7.2

Literale

**U-Suffix für unsigned
Konstanten**

Required
entscheidbar · Single TU

7.3

Literale

Kein kleines l als Suffix

Required
entscheidbar · Single TU

8.1

Deklarationen

**Typen explizit angeben
(nur C90)**

Required
entscheidbar · Single TU

9.1

Initialisierung

**Nicht lesen, bevor
gesetzt**

**Mandatory
unentscheidbar · System**

Ablauf: Grundbegriffe → Regeln 5.1 bis 9.1 → Zusammenfassung → Projektidee

Grundbegriffe: So liest man eine Regel



Kategorie

Mandatory

Immer einhalten, keine Abweichung erlaubt.

Required

Einhalten; Abweichung nur mit formaler Begründung.

Advisory

Empfehlung, so weit wie praktikabel befolgen.



Entscheidbarkeit

Entscheidbar

Ein Tool kann immer sicher „ja“ oder „nein“ sagen, z. B. bei Namen oder Suffixen.

Unentscheidbar

Hängt von Laufzeitwerten oder dem Kontrollfluss ab. Tools können nur schätzen.



Analyse-Umfang

Single Translation Unit

Jede .c-Datei lässt sich einzeln prüfen.

System

Alle Dateien des Projekts müssen gemeinsam analysiert werden.

5.1

Externe Bezeichner müssen unterscheidbar sein

Required

Entscheidbar

System

C90, C99

- Globale Namen müssen sich innerhalb der Zeichen unterscheiden, die Compiler und Linker tatsächlich auswerten.
- C90: nur 6 Zeichen garantiert, Groß-/Kleinschreibung egal
- C99: 31 Zeichen garantiert
- Viele Toolchains können mehr, portabel ist aber nur das Minimum.



Gefahr bei Nichtbeachtung

Undefiniertes Verhalten: Der Linker kürzt beide Namen auf dasselbe Symbol. Ergebnis ist ein Linkerfehler oder zwei Variablen teilen sich eine Speicherstelle.

```
/* schlecht: erste 31 Zeichen gleich */  
int32_t sensor_temperature_front_left_raw_value;  
int32_t sensor_temperature_front_left_raw_valid;
```

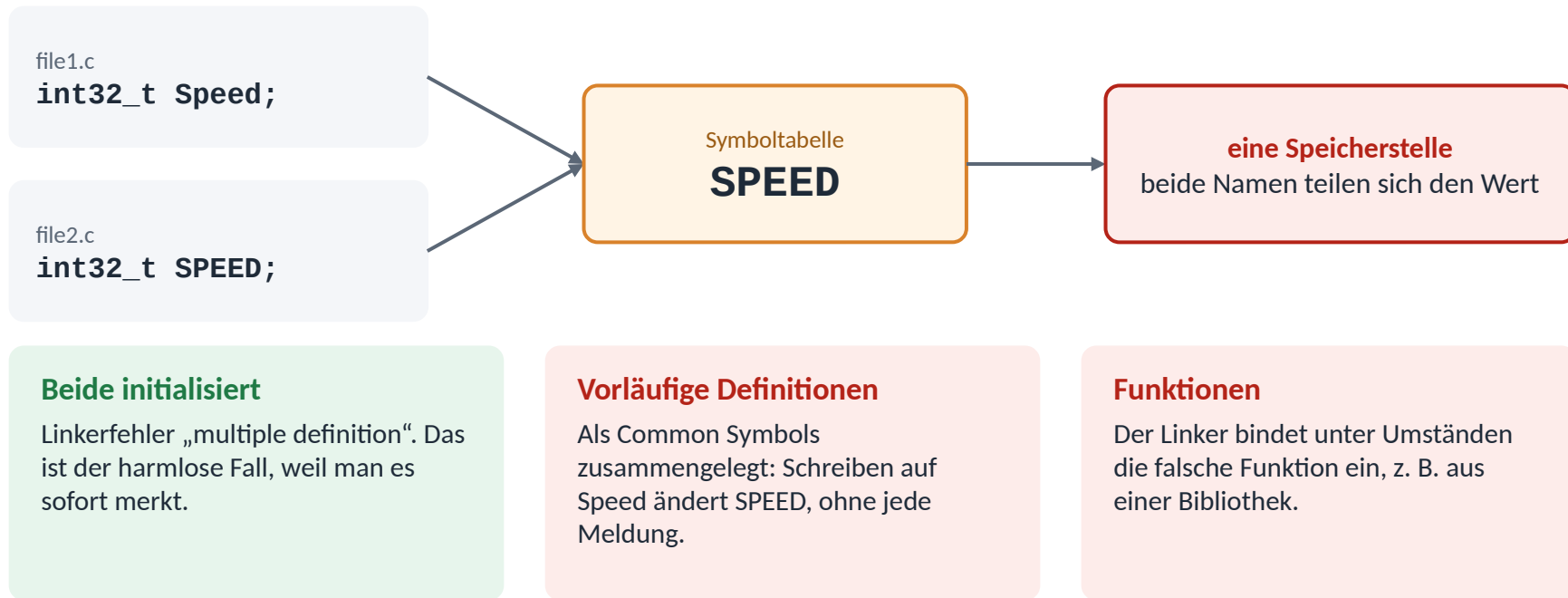
```
/* schlecht bei C90-Minimum (6 Zeichen) */  
/* file1.c */ int32_t Speed;  
/* file2.c */ int32_t SPEED;
```

```
/* gut: kurz und früh unterscheidbar */  
int32_t temp_fl_raw;  
int32_t temp_fl_valid;
```

5.1

Was im Linker passiert

Beispiel: Linker wertet nur 6 Zeichen aus, Groß-/Kleinschreibung egal (C90-Minimum)



Moderne ELF-Linker haben praktisch kein Längenlimit, GCC ab Version 10 nutzt `-fno-common`. Das Risiko betrifft vor allem ältere Embedded-Toolchains.

5.2

Namen im gleichen Scope und Namensraum unterscheidbar

Required

Entscheidbar

Single TU

C90, C99

- Gegenstück zu 5.1 für nicht-externe Namen, z. B. static-Variablen oder lokale Variablen.
- Mindestgrenzen: C90 31 Zeichen, C99 63 Zeichen
- Externe Namen fallen unter 5.1, Makros unter 5.4 und 5.5.
- Gleicher Name in einem anderen Scope: kein Verstoß gegen 5.2, aber gegen 5.3.

**Gefahr bei Nichtbeachtung**

Undefiniertes Verhalten: Ein Compiler mit kurzer Grenze sieht zwei gleiche Namen und meldet eine Redefinition oder behandelt beide als eine Variable. Vor allem ein Portierungsproblem.

```
/* schlecht (31 signifikante Zeichen) */
static uint16_t
    motor_controller_setpoint_speed_rpm_left;
static uint16_t
    motor_controller_setpoint_speed_rpm_right;

/* gut */
static uint16_t motor_set_rpm_left;
static uint16_t motor_set_rpm_right;

void f(void)
{
    /* ok für 5.2, verletzt aber 5.3 */
    uint16_t motor_controller_setpoint_speed_x;
}
```

5.3

Kein Verdecken äußerer Bezeichner (Shadowing)

Required

Entscheidbar

Single TU

C90, C99

- Ein Name in einem inneren Scope darf keinen Namen aus einem äußeren Scope verdecken.
- Reihenfolge: Datei-Scope → Block → verschachtelter Block
- Verschiedene Namensräume, z. B. struct-Tag und Variable, verdecken sich nicht.

Gefahr bei Nichtbeachtung

Gültiges C: Der Compiler nimmt still die innerste Deklaration. Die globale Variable wird nie aktualisiert, z. B. bleibt ein Fehlerzähler auf 0 und eine Abschaltung löst nie aus. GCC und Clang warnen nur mit `-Wshadow`, das nicht in `-Wall` oder `-Wextra` enthalten ist.

```
uint16_t error_count = 0U;

void check_sensors(void)
{
    uint16_t error_count = 0U; /* schlecht */
    if (sensor_fault())
    {
        error_count++; /* nur lokale Kopie! */
    }
}

uint16_t speed;
void fn(void)
{
    typedef float32_t speed; /* schlecht */
}

/* gut: eindeutiger Name, z.B. local_errors */
```

5.8

Namen mit externer Bindung müssen eindeutig sein

Required

Entscheidbar

System

C90, C99

- Ein Name für ein globales Objekt oder eine globale Funktion darf im ganzen Projekt für nichts anderes verwendet werden.
- Das gilt auch für static-Funktionen in anderen Dateien und für lokale Variablen.
- Namen ohne Bindung dürfen untereinander mehrfach vorkommen.

 Gefahr bei Nichtbeachtung

Jede Datei ist für sich korrekt, deshalb meldet der Compiler nichts. Die falsche Funktion wird aufgerufen oder geändert, Debugger und Map-File verwirren. Entfernt jemand später das static, gibt es einen Linkerfehler.

```
/* file1.c */
int32_t count;           /* externe Bindung */
void log_event(void)     /* externe Bindung */
{
}

/* file2.c */
static void log_event(void) /* schlecht */
{
    int16_t count;         /* schlecht: Kollision */
    int16_t index;         /* ok: keine Bindung */
}
```

7.2

Unsigned-Konstanten brauchen ein U-Suffix

Required

Entscheidbar

Single TU

C90, C99

- Jede Ganzzahlkonstante mit vorzeichenlosem Typ bekommt u oder U, auch in #if und #elif.
- Der Typ hängt ab von: Größe des Werts, int-Breite, Suffix und Zahlenbasis
- Nur der Typ der Konstante selbst zählt, nicht der Kontext.



Gefahr bei Nichtbeachtung

Dieselbe Konstante hat je nach Plattform einen anderen Typ. Vergleiche, Bitmasken und Shifts verhalten sich dann still anders (Beispiel auf der nächsten Folie).

Beispiel: 16-Bit int, 32-Bit long

Konstante	Typ	Bewertung
32767	signed int	konform
0x7fff	signed int	konform
32768	signed long	konform
32768u	unsigned int	konform
0x8000	unsigned int	nicht konform
0x8000u	unsigned int	konform

7.2

Gleicher Code, anderes Ergebnis

Typische Falle beim Portieren von 16 auf 32 Bit

```
int16_t s = -1;  
if (s < 0x8000) /* wahr oder falsch? */
```

16-Bit int

FALSCH

- 0x8000 passt nicht in int → unsigned int
- s wird nach unsigned umgewandelt: -1 → 65535
- 65535 < 32768

32-Bit int

WAHR

- 0x8000 passt in int → signed int
- keine Umwandlung nötig
- -1 < 32768



Mit **0x8000U** ist die Konstante überall unsigned: gleiches Verhalten auf allen Plattformen, und die Absicht ist im Code sichtbar.

7.3

Kein kleines „l“ als Literal-Suffix

Required

Entscheidbar

Single TU

C90, C99

11l

liest sich wie 111

11L

eindeutig

```
const int64_t a = 11l;    /* schlecht */
const int64_t b = 11L;    /* gut */

const uint64_t c = 0Ull;  /* schlecht */
const uint64_t d = 0ULL;  /* gut */
```

- Für long bzw. long long immer L bzw. LL schreiben, nie l oder ll.
- Für den Compiler ist beides identisch, es geht rein um Lesbarkeit.
- Leicht zu befolgen und automatisch prüfbar.

**Gefahr bei Nichtbeachtung**

Kein Compilerproblem, sondern ein menschliches: l und 1 sind in vielen Schriften kaum unterscheidbar. Ein Grenzwert von 111 statt 11 rutscht leicht durch Review oder Abtippen.

8.1

Typen müssen explizit angegeben werden

Required

Entscheidbar

Single TU

nur C90

- C90 nimmt bei fehlendem Typ automatisch int an („implicit int“).
- Betrifft: Objekte, Parameter, struct-Member, typedefs, Rückgabetypen
- Ab C99 verboten: moderne Compiler brechen ab, GCC ab Version 14 standardmäßig.

Gefahr bei Nichtbeachtung

Man bekommt einen anderen Typ als gedacht: const k hinter char c ist const int. Passt die Deklaration nicht zur echten Definition (z. B. uint8_t), ist das Verhalten undefiniert: falsche Werte oder Absturz.

```
extern x;                                /* schlecht */
extern f(void);                          /* schlecht */
void g(char c, const k);                 /* schlecht */
struct s { const b; };                  /* schlecht */

extern int16_t x;                         /* gut */
extern int16_t f(void);                   /* gut */
void g(char c,
          const int16_t k);               /* gut */
struct s
{
    const int16_t b;                      /* gut */
};
```

8.1

Beispiel: Pointer als impliziter int (64-Bit)

int hat 4 Byte, ein Pointer 8 Byte: die oberen 32 Bit gehen verloren

```
/* file1.c */
uint8_t *get_buffer(void)
{
    static uint8_t buf[64];
    return buf;
}

/* file2.c */
extern get_buffer();    /* impliziter int */

void use(void)
{
    uint8_t *p = (uint8_t *)get_buffer();
    p[0] = 42U;          /* Absturz hier */
}
```

1 • RAX: Rückgabe von get_buffer()

00007F3A

12345678

2 • Aufrufer liest nur EAX (int, 32 Bit)

- - - - -

12345678

3 • Cast zurück auf Pointer (64 Bit)

00000000

12345678

Lesen als int: kein Absturz, nur eine abgeschnittene Zahl.

Dereferenzieren: Zugriff auf eine falsche Adresse → Segfault oder stille Speicherkorruption.

Auf 32-Bit: int und Pointer je 4 Byte → funktioniert zufällig.

9.1

Nicht lesen, bevor der Wert gesetzt wurde

Mandatory

Unentscheidbar

System

C90, C99

- Lokale Variablen (automatische Speicherdauer) haben keinen Startwert.
- Array-Elemente und struct-Member zählen jeweils einzeln.
- Globale und static-Variablen werden automatisch mit 0 initialisiert.
- Mandatory: keine Abweichung erlaubt

Gefahr bei Nichtbeachtung

In der Variable steht Stack-Müll, anders bei jedem Lauf. Debug- und Release-Build verhalten sich unterschiedlich. Der Optimierer darf das undefinierte Verhalten ausnutzen und Code entfernen. Alte Stack-Daten können Geheimnisse preisgeben.

```
void read_sensor(bool_t ok, uint16_t *out)
{
    if (ok) { *out = adc_read(); }
}

void task(void)
{
    uint16_t value;
    read_sensor(false, &value);
    if (value > LIMIT)      /* schlecht */
    {
        shutdown();
    }
}

void jump(void)
{
    goto L1;
    uint16_t x = 10U;      /* übersprungen */
L1:
    x = x + 1U;            /* schlecht */
}
```

9.1

Warum unentscheidbar? Das Halteproblem

Beweisskizze durch Widerspruch: Gegenteil annehmen → Widerspruch → Folgerung für 9.1

1 Annahme: Es gibt eine perfekte Prüf-Funktion

```
bool_t halts(program p, input i);
/* true: p endet, false: p läuft endlos */
```

2 Programm, das immer das Gegenteil macht

```
void paradox(program p)
{
    if (halts(p, p)) /* endet p(p)? */
    {
        for (;;) { } /* dann: endlos */
    }
    else
    {
        return; /* sonst: sofort enden */
    }
}
```

3 Aufruf paradox(paradox)

halts sagt	paradox macht	Ergebnis
„endet“	läuft endlos	halts falsch
„läuft endlos“	endet sofort	halts falsch

Jede Antwort ist falsch → halts kann es nicht geben.

4 Brücke zu 9.1

```
uint16_t x;
P(); /* beliebiges Programm */
y = x; /* Verstoß genau dann, wenn P endet */
```

Ein perfekter 9.1-Prüfer würde halts beantworten, kann es also nicht geben. Tools müssen schätzen: Fehlalarme oder übersehene Verstöße.

Zusammenfassung



Undefiniert oder plattformabhängig

5.1

5.2

7.2

8.1

9.1

Gleicher Quellcode, anderes Verhalten je nach Compiler, Linker oder Plattform. Bei 8.1 vor allem durch Typ-Mismatch.



Definiert, aber unerwartet

5.3

5.8

Der Compiler macht alles „richtig“, aber anders, als der Programmierer denkt.



Reine Lesbarkeit

7.3

Der erzeugte Code ist identisch. Fehler entstehen beim Lesen und im Review.

Kernaussage: Fast alle Verstöße übersetzt der Compiler ohne Fehlermeldung. Statische Analyse ist deshalb Pflicht.

Teil 2

Projektidee